
University of
Nottingham
UK | CHINA | MALAYSIA
Attendance
Developing
Maintainable
Software
COMP2013 – Lecture 03
Marjahan Begum and Max L. Wilson

Meme of the Week!

- Meme of the week is completely optional, and for run
- Meme of the week reminds us what we learned last week though

- *Task: submit a funny software engineering meme that most closely represents the learning outcome of the week*

- Max will rank the best memes submitted every week
  - 1st = 5 points, 2nd = 3pts, 3rd = 2pts, runner ups = 1pt
- These are the best memes submitted last week

Today's Lecture Slot

Introduction : Identifying and fixing bad code

40mins

Industry Interview with Mustafa Hamada

DevOps in practice

30mins

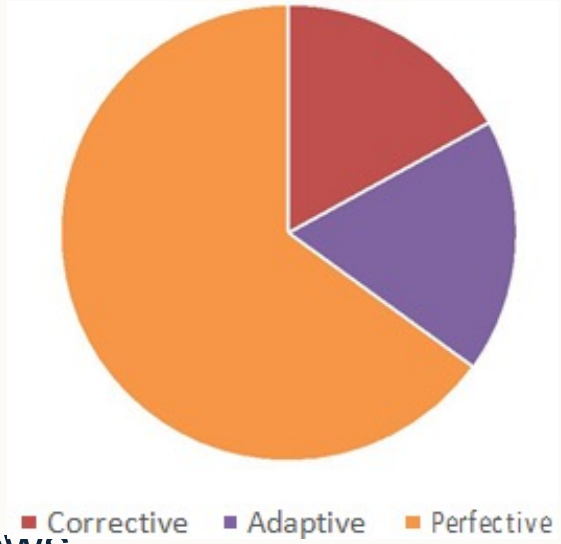
Lab 03
Continuation of Refactoring

20mins

- Modularity – can we divide the code in modular form?
- Reusability – can we reuse the same module/code?
- Analyzability – is it easy to analyze the code?
- Changeability – is it easy to change the code?
- Modification Stability – will it affect other code?
- Testability – how can we test the code?
- Compliance – does the code works in different OS or version?

3 Main Categories of Maintenance

(or 4, depending on authors)



- Corrective Maintenance
  - Finding and fixing errors in the system
    - Example: Fixing bugs
- Adaptive Maintenance
  - Adapting to changes in the environment
    - Example: Updating for new version of windows
- Perfective + Preventive Maintenance
  - Stakeholders have new or changed requirements
  - Ways are identified to increase quality and prevent future problems

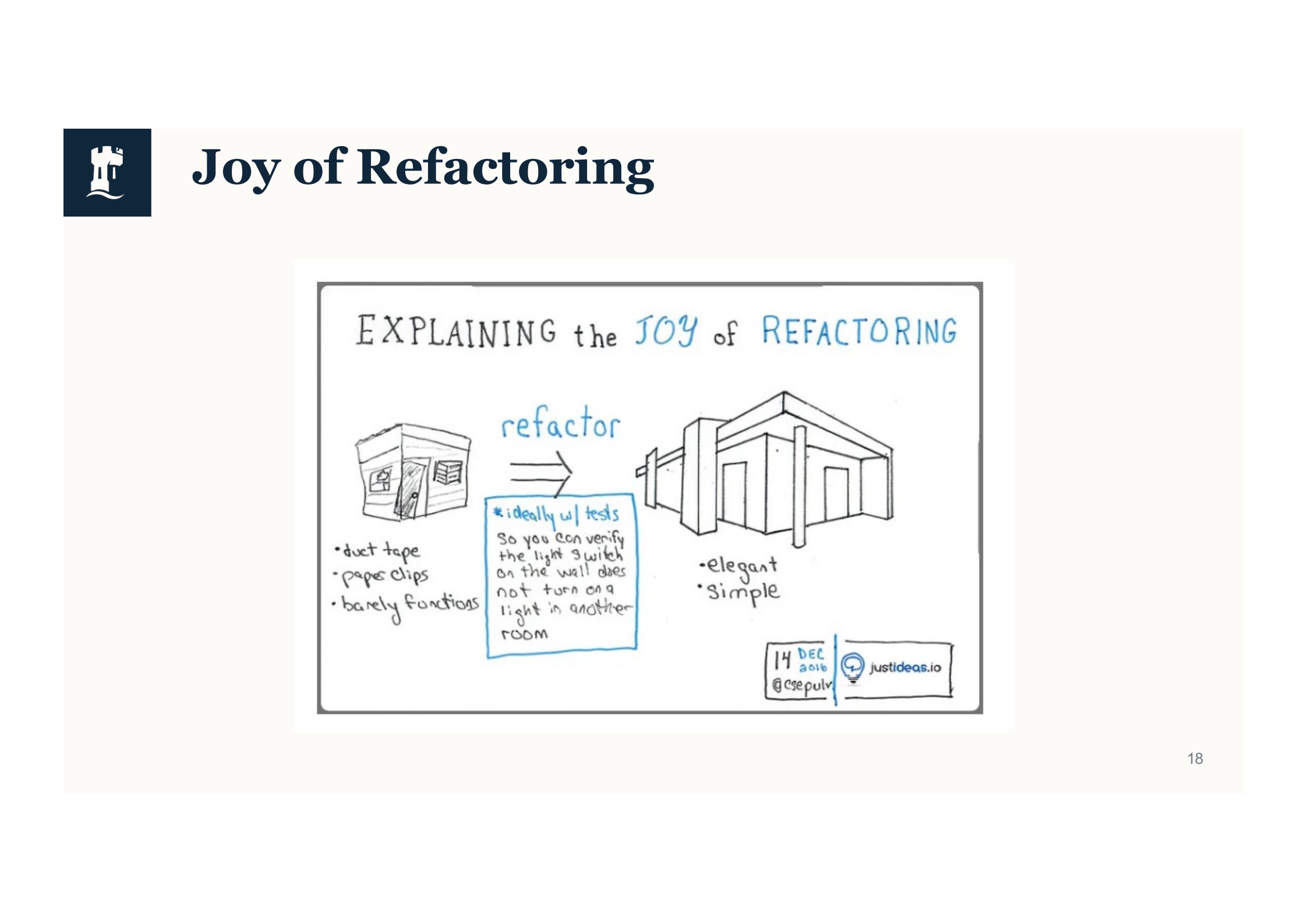
Legacy Code

Legacy Code Definitions

- Traditional Definition:
  - Code that is old;
  - Built with outdated technology (and still in use);
  - System that is difficult to modify or replace due to design or technology stack.
- The Pragmatic Definition (popularised by Michael Feathers):
  - Legacy code is simply code maybe without tests, where any change is inherently risky;
  - You cannot be certain you have not broken existing functionality.
  - In principle, legacy code is code that is difficult and risky to change.

Legacy Code Characteristics

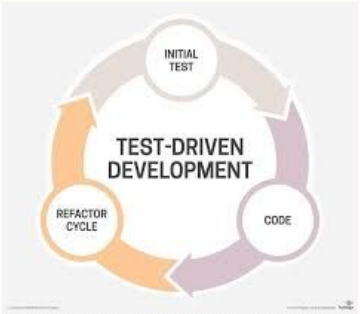
- Lack tests: No automated unit, integration, or end-to-end tests.
- Poor documentation: Missing, incorrect, not meaningful comments and docs (e.g. UML, IEEE
- Standard for Software and System Test Documentation)
- Outdated technology: Using languages or frameworks that are no longer supported.
- “Big Ball of Mud” architecture: Poorly structured; tightly coupled and low cohesion; hard to separate changes (without changes to other codes).
- Knowledge silos: Only small number (one or two) of people (who may have left the company) understand how it works.
- Fragile: A small change in one part of the system causes unexpected bugs in unrelated parts.



Joy of Refactoring

The Software Maintenance Starter Pack

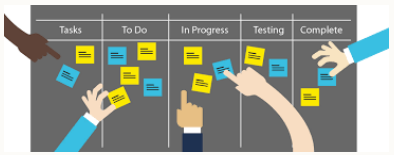
GitHub



Gradle

rm –rf *

"I've just copied my code from ChatGpt…its bound to work"



The Software Maintenance Starter Pack –DevOps

Docker

GitLab

Kubernetes

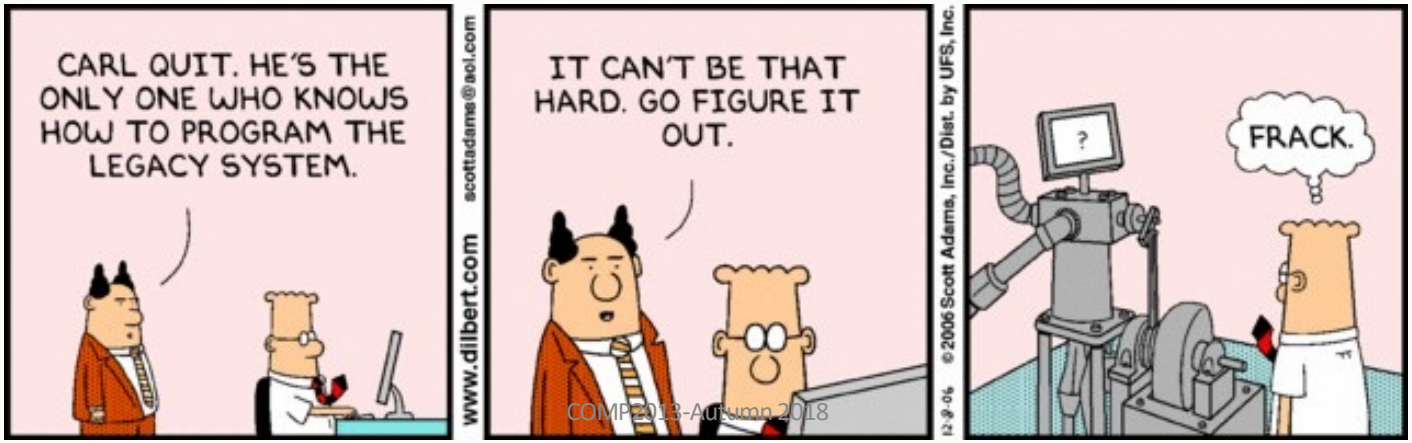
```
# What does YAML mean?
YAML:
    - Y: YAML
    - A: Ain't
    - M: Markup
    - L: Language
```

Jenkins

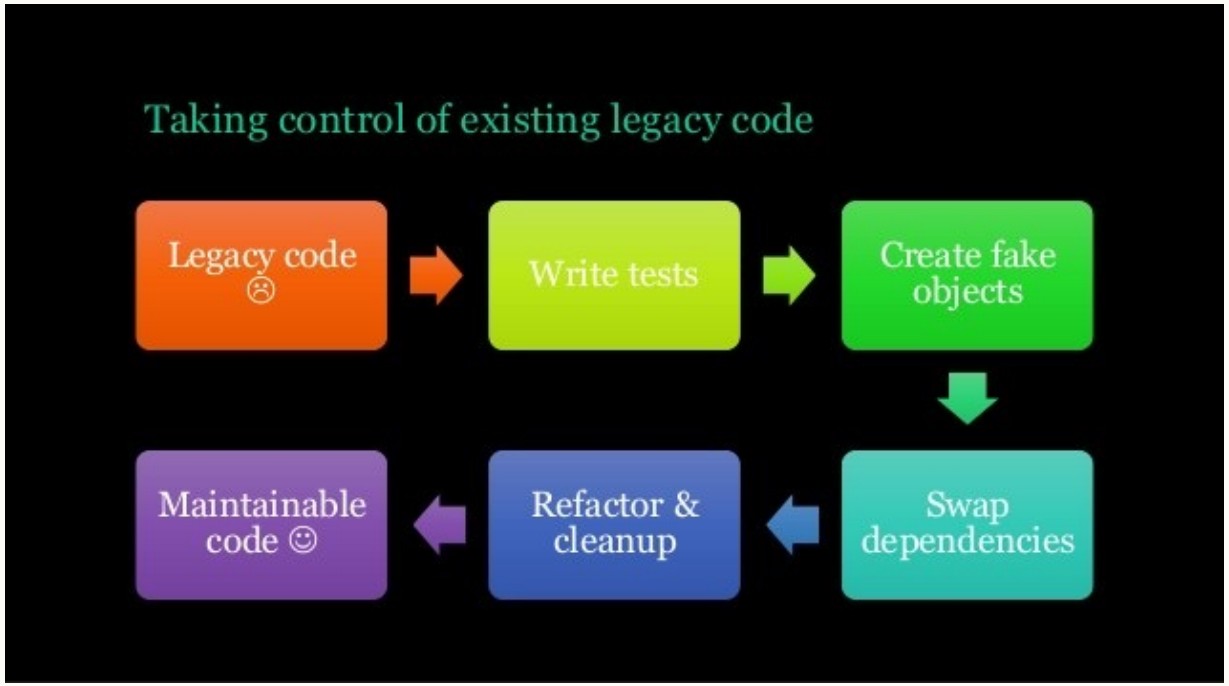
Some issues with DevOps adaptation

- Legacy systems proprietary configuration languages and frameworks.
- Issues with test automations

Legacy Code Maintenance The Core Of An SE Role

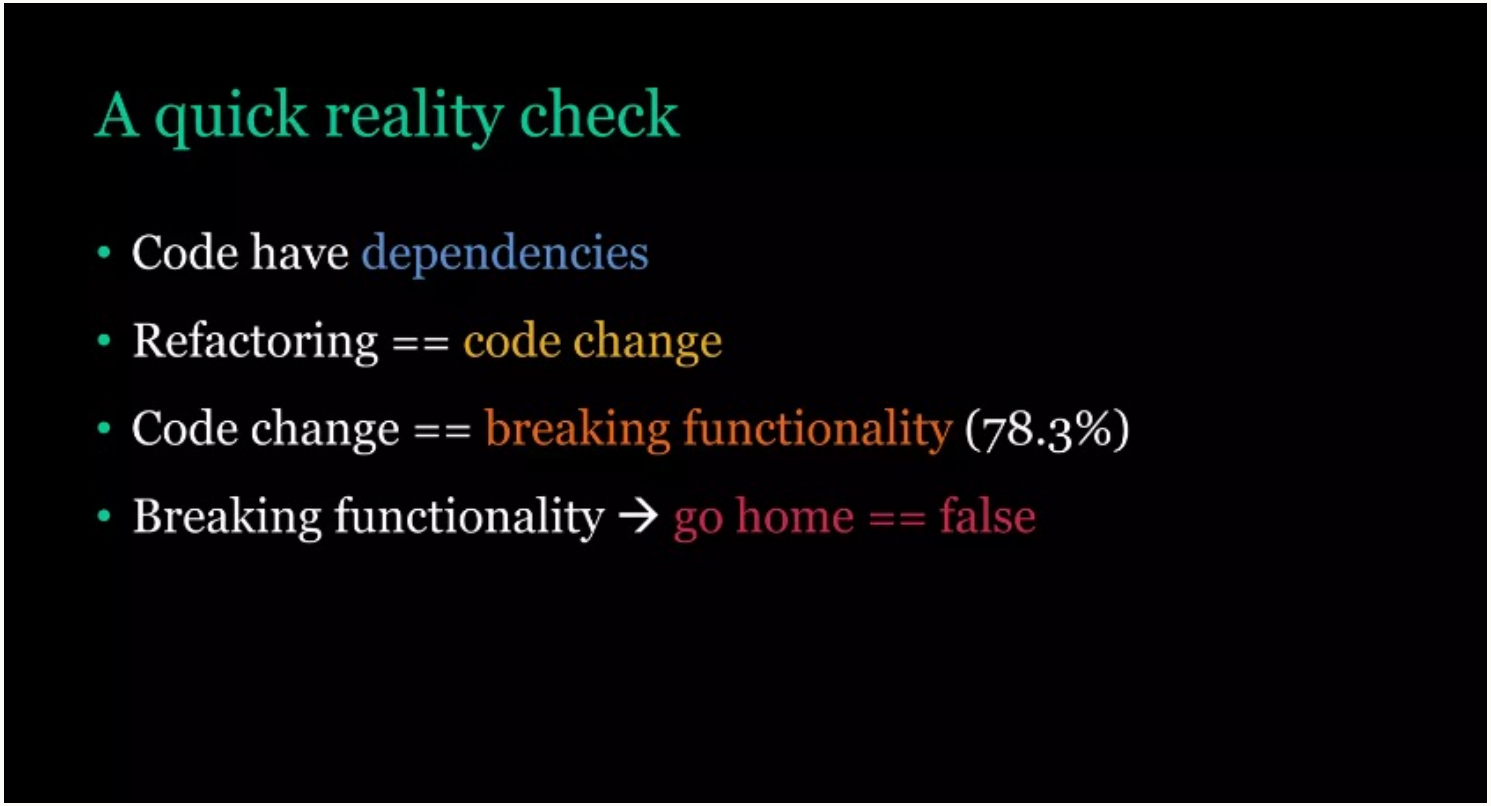


The Process of Working With Legacy Code



Still confused? Try this: https://www.slideshare.net/dhelper/working-with-c-legacy-code

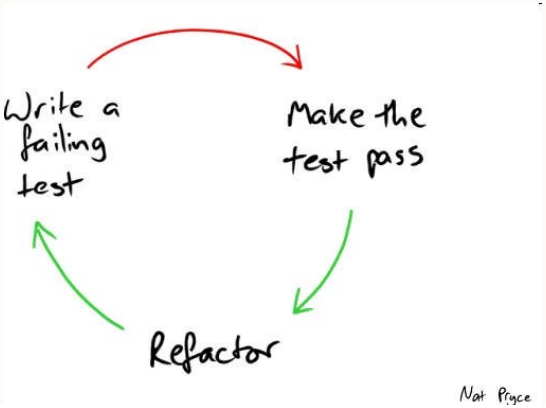
Reality Reality check!

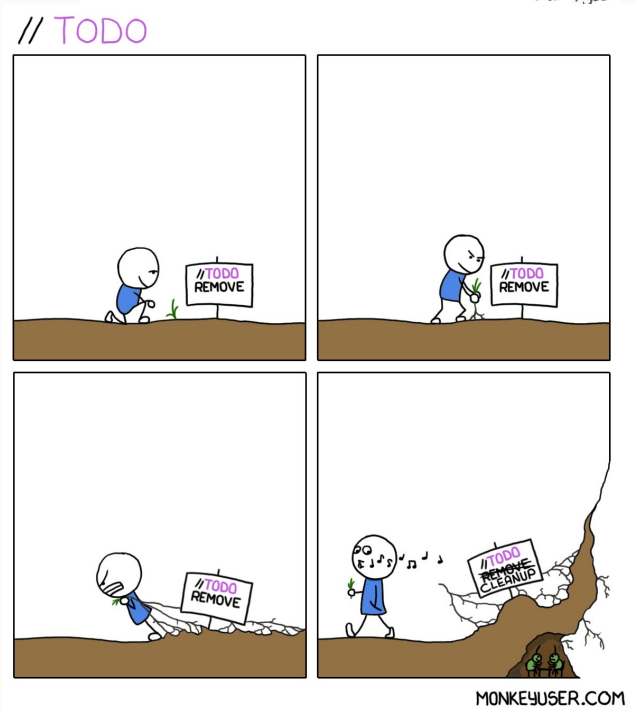


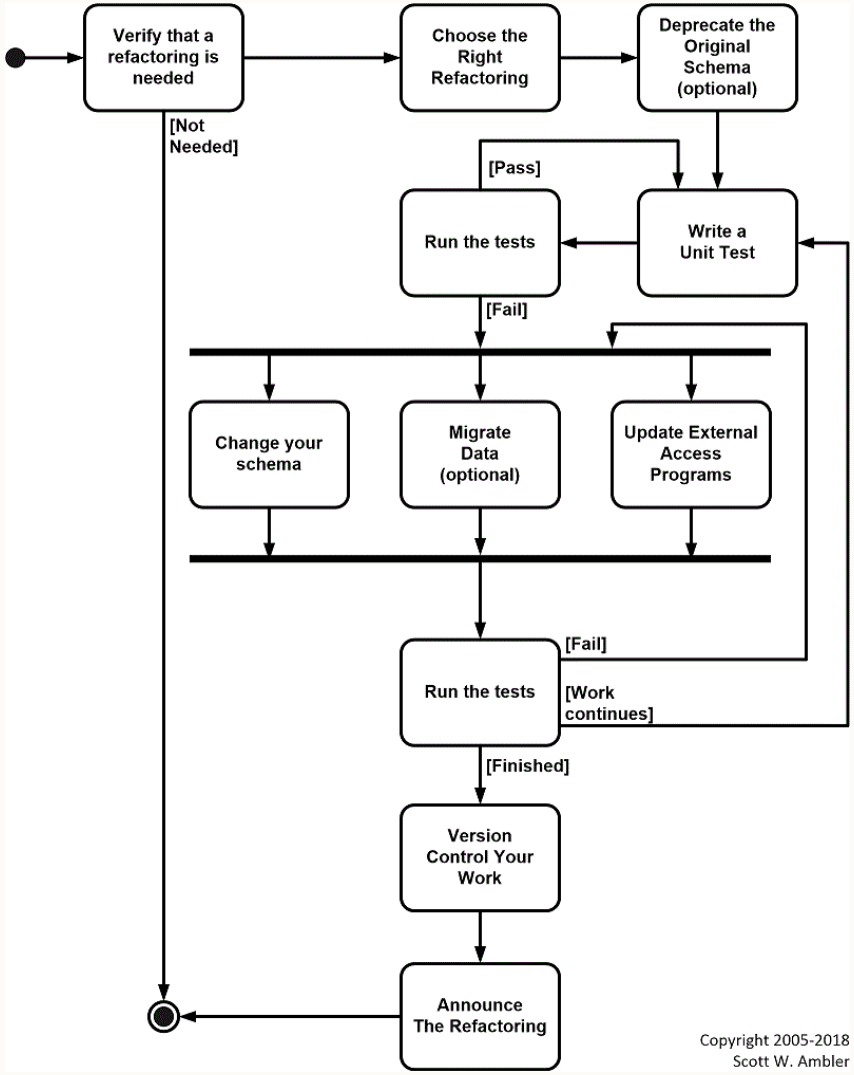
Source : https://www.slideshare.net/dhelper/working-with-c-legacy-code

Refactoring

Easy in theory







Quotes about refactoring from the authorities

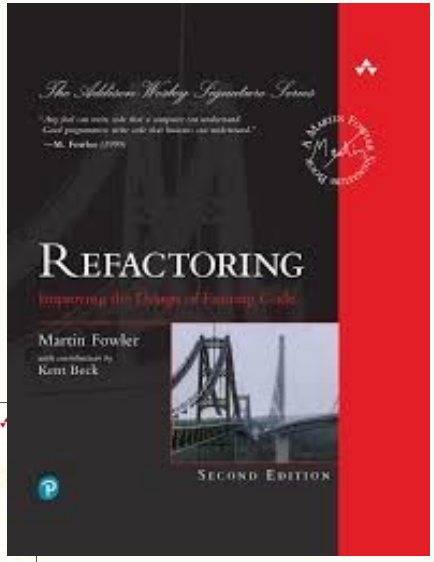
- *"A series of small decisions and actions all made through the filter of a set of values and the desire to make something excellent"* – Chad Fowler, Ruby Central

- *"Refactoring is the process of changing a software system in a way that does not alter the external behavior of the code yet improves its internal structure"* – Martin Fowler

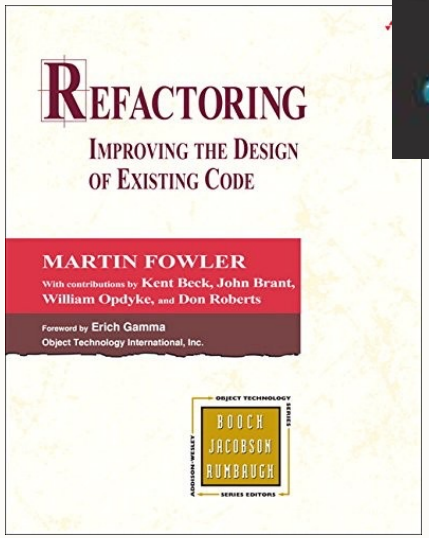
- *"The XP philosophy is to start where you are now and move towards the ideal. From where you are now, could you improve a little bit?"* - Kent Beck

- *"Any fool can write code that a computer can understand. Good programmers can write code that humans can understand"* – Martin Fowler

Martin Fowler
Refactoring Godfather



Kent Beck
Agile Guru &
Code Smeller



Refactoring improves existing code

- The process of changing a software system to **not alter the external behavior** of the code yet improves **its internal** structure in an incremental fashion
- A disciplined way to clean up code to minimize the chance of introducing bugs
- Improving the design of code after it has been written
  - Can be code you wrote yesterday or code written decades ago
- Explores the **tradeoff** space between clarity and performance
- Refactoring is not optimizing code to make it run faster
  - It is about making code make more sense and increasing its robustness
  - Making code open for modification
  - Application of SOLID principles and design pattern

Kent Beck Says…

"The majority of the cost of software is incurred after the software has been first deployed. Thinking about my experience of modifying code, I see that I spend much more time reading the existing code than I do writing new code. If I want to make my code cheap, therefore, I should make it easy to read."

```
/**
* Code Readability
*/
if (readable()) {
    be_happy();
} else {
    refactor();
}
```

A Trivial HTML Example Of Using Whitespace in Code

```
<html> <head> <title>Make your HTML code readable by using white space</title> </head> <body> <p>Do you want an easy way to understand your code? If yes, use white space to organize your code. You can use white space as much as possible to make your code readable. You will be surprised how easy it is to follow your code when you use appropriate amount of white space. If you do not use white space, your HTML code will look messy. So: </p> <ul> <li>group related tags and separate them by indenting</li> <li>use white space</li> <li>use comments when necessary to explain what your code does</li> </ul> </body> </html>
```

```
<html>
  <head>
    <title>
      Make your HTML code readable by using white space
    </title>
  </head>
  <body>
    <p>
      Do you want an easy way to understand your code? If yes, use white space to organize your code. You can use white space as much as possible to make your code readable. You will be surprised how easy it is to follow your code when you use appropriate amount of white space. If you do not use white space, your HTML code will look messy. So:
    </p>
    <ul>
      <li>group related tags and separate them by indenting</li>
      <li>use white space</li>
      <li>use comments when necessary to explain what your code does</li>
    </ul>
  </body>
</html>
```

Who is involved in Refactoring?

- Refactoring is a multidisciplinary activity involving:
  - Programmers/Developers
  - Senior designers
  - Management
  - System Architects
- Each of these roles will adapt the principles of refactoring to a specific project or workspace
- Refactoring occurs in different forms across a project

Refactoring Roadmap

Quick Wins

- Remove dead code
- Remove code duplicates
- Enhance identifier naming
- Reduce method size

Divide & Conquer

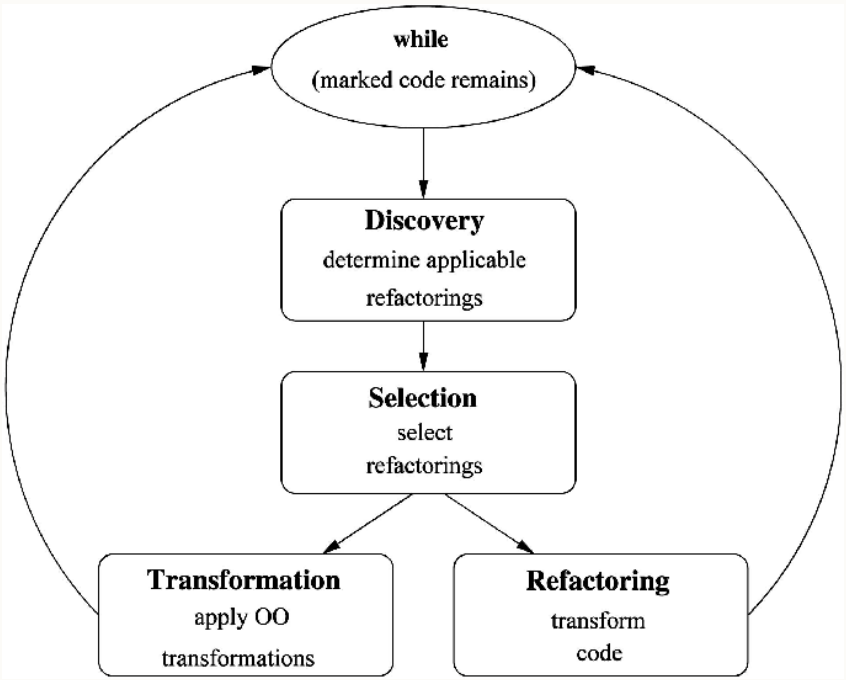
- Discover & split code into components
- Enhance component encapsulation
- Reduce coupling

Inject Quality In

- Cover components with automated tests
- Enhance components internal design

Continuous Review

Sustainable Refactoring Roadmap - v1.2



Build Your Test Scaffold Before You Touch Anything

- The key to successful refactoring is to **AVOID** breaking anything while you make your changes
- This requires that you use the existing or develop a new test scaffold
  - Regression testing, right?
- Run the test scaffold after every instance of change, not after every feature refactored
- **Legacy code may not include any test cases, in which case you need to add them before you start 'tinkering' i.e. making incremental changes**
- Substitute 'live' components with dummy ones i.e. use '**Mock Objects**'
- Once you have done this you are ready to go
- Still Confused? Try this: https://martinfowler.com/articles/mocksArentStubs.html

Mock Objects

- A mock object is a simulated version of a genuine object (a dependency) used during testing.
- Configured to replicate the real object's behaviour in a controlled manner.
- Can isolate the code under test and verify its interactions with thosedependencies.

- https://realpython.com/python-mock-library/

Mock Objects

Create Interfaces/Classes to Mock

```
// UserService.java
public interface UserService {
        String getUserName(String userId);
        int getUserAge(String userId);
}

// EmailService.java
public interface EmailService {
  void sendEmail(String recipient, String message);
}
```

Create Class Under Test

```
public class UserManager {
private final UserService userService;
private final EmailService emailService;

public UserManager(UserService userService, EmailService emailService) {
  this.userService = userService;
  this.emailService = emailService;
}

public String getUserDetails(String userId) {
  String name = userService.getUserName(userId);
  int age = userService.getUserAge(userId);

return String.format("Name: %s, Age: %d", name, age);
}

public void notifyUser(String userId) {
  String userName = userService.getUserName(userId);
  emailService.sendEmail(userName, "Your account has been updated!");
}
```

Mock Objects

```
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

class UserManagerTest {

    @Test
    void testGetUserDetails() {
        UserService mockUserService = mock(UserService.class);
        EmailService mockEmailService = mock(EmailService.class);

        when(mockUserService.getUserName("123")).thenReturn("John Doe");
        when(mockUserService.getUserAge("123")).thenReturn(30);

        UserManager userManager = new UserManager(mockUserService, mockEmailService);
        String result = userManager.getUserDetails("123");

        assertEquals("Name: John Doe, Age: 30", result);

        verify(mockUserService).getUserName("123");
        verify(mockUserService).getUserAge("123");
    }
    …
}
```

Using `mock()` to create mock implementations of dependencies.

Using `when().thenReturn()` to define mock behaviour.

Using `verify()` to check if mock methods were called.

Mock Objects

```
class UserManagerTest {
    …
    @Test
    void testNotifyUser() {
        UserService mockUserService = mock(UserService.class);
        EmailService mockEmailService = mock(EmailService.class);

        when(mockUserService.getUserName("456")).thenReturn("jane@example.com");

        UserManager userManager = new UserManager(mockUserService, mockEmailService);
        userManager.notifyUser("456");

        // Verify email service was called with specific parameters
        verify(mockEmailService).sendEmail("jane@example.com", "Your account has been updated!");
    }

    @Test
    void testUserNotFound() {
        UserService mockUserService = mock(UserService.class);
        EmailService mockEmailService = mock(EmailService.class);

        when(mockUserService.getUserName("999")).thenThrow(new IllegalArgumentException("User not found"));

        UserManager userManager = new UserManager(mockUserService, mockEmailService);

        assertThrows(IllegalArgumentException.class, () -> {
            userManager.getUserDetails("999");
        }); }}
```

Using `thenThrow()` to test error conditions.

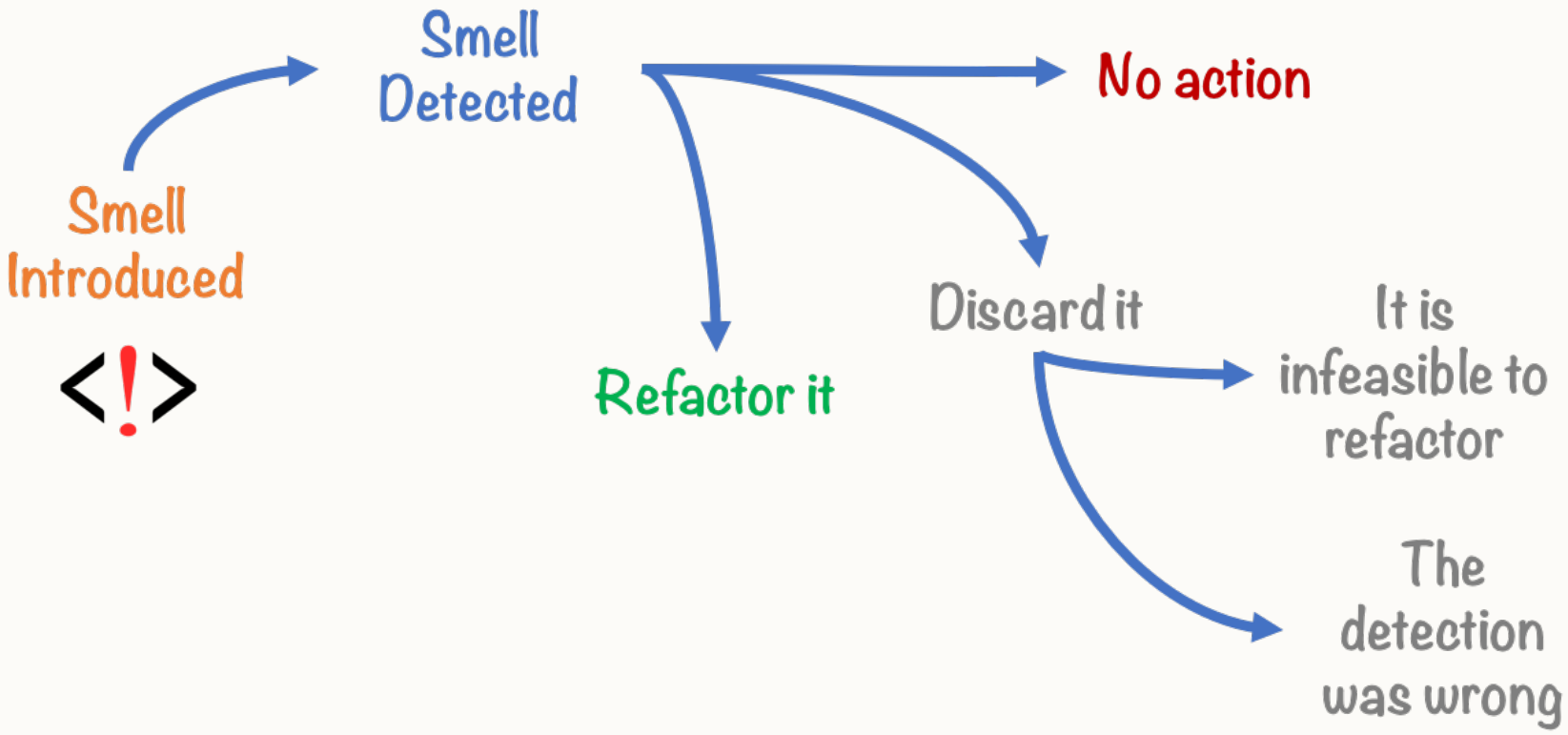
Mock Objects on Legacy Code

- Mocks are the key to breaking this deadlock:
  - You cannot test code that interacts with a database? **Mock the database**.
  - You cannot test code that calls a third-party API? **Mock the API client**.
  - You cannot test code that reads from the file system? **Mock the file reader**.
- Isolate small segments of even the most tangled legacy code and begin building a test suite around them.
- Making it cleaner and more maintainable.
- Mock objects are a fundamental technique for writing effective unit tests, which are themselves the primary antidote to legacy code.

Identify Code Smells

Decide what you are going to refactor

What is Code Smell?



What is Code Smell?

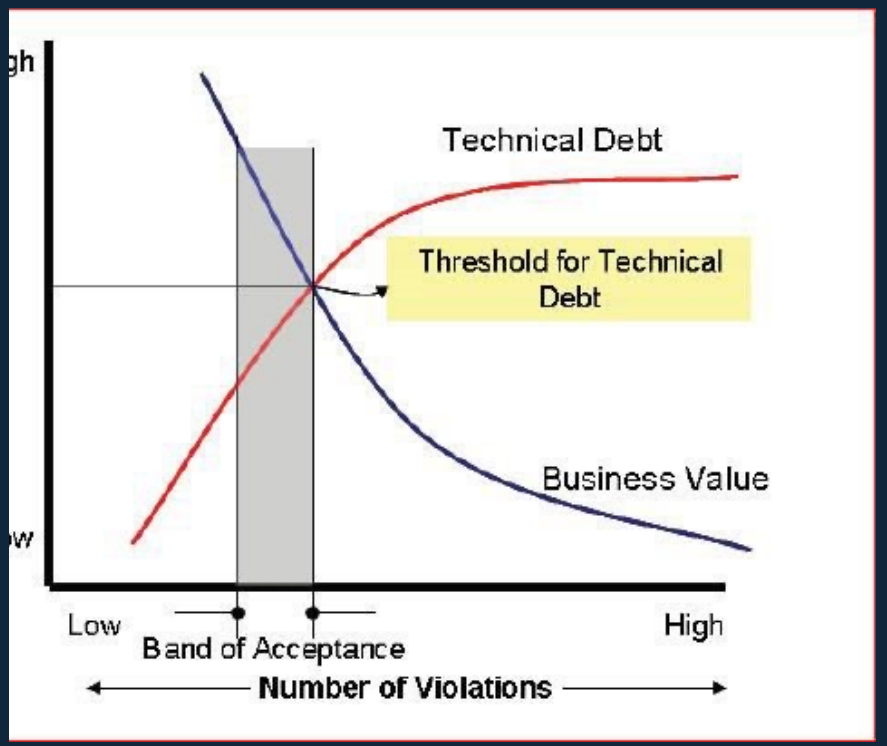
- Code smells are **categories of issues in code** that may give rise to deeper problems over time.
- Towards warning signs, indicating potential poor coding practices.
- **Not errors or bugs and do not impair the software's functionality**.
  - Reveal weaknesses in design that can slow down development or increase the risk of faults or failures further down the line.
- Code smells are inherently **subjective**.
  - One developer regards as a smell, but another might well see as a perfectly reasonable solution.
  - Decades of collective experience and often point to violations of fundamental software design principles.

Why Code Smell Matters?

- They act as early warning signs that your code may be:
  - **Hard to understand**: It becomes difficult for other developers (or your future self) to read.
  - **Hard to change**: A simple change can require modifications in numerous places.
  - **Hard to test**: Tightly coupled code is almost impossible to unit-test properly, linking back to our legacy-code discussion.
  - **Bug-prone**: complex, tangled code is more likely to hide subtle faults.

Why Code Smell Matters?

- Technical debt compounds as developers compensate for smelly code without actually doing anything to eliminate it.



Code Smells vs Bugs

Item	Bugs	Code Smells
Definition	Flaws in code that cause incorrect behaviour, crashes or unexpected outputs.	Indicators of poor design that may lead to future bugs or maintenance issues.
Characteristics	•Cause visible errors during execution. •Can be reproduced with specific inputs. •Need immediate fixing. (High: Must fix) •Clearly right or wrong behaviour.	•Do not necessarily cause immediate issues. •Based on best practices and experience. •Addressed to avoid future problems. (Medium: Should fix) •Long-term code healthcare.

Code Smells vs Bugs

- Fix **BUGS** to make the software work correctly now.
- Address **CODE SMELLS** to make the software maintainable for the future.
- Code smells often create an environment where bugs are more likely to be introduced.
- Good developers fix bugs;
- Great developers also eliminate code smells that could cause future bugs

Code Smells Because Its Hard to Understand (not because it is incorrect)

- Duplicated Code:
  - bad because if you modify one instance of duplicated code but not the others, you (may) have introduced a bug!
- Long Methods:
  - long methods are more difficult to understand
  - Note: performance concerns with respect to lots of short methods are largely unfounded
- Large Classes:
  - classes try to do too much, which reduces cohesion (violates single responsibility, “The Blob”
- Long Parameter Lists:
  - hard to understand, can become inconsistent
- Divergent Changes:
  - related to cohesion where one type of change requires changing one subset of methods; another type of change requires changing another subset

More code smells examples

- Lazy Classes:
  - A class that no longer “pays its way”, part formed functionality
- Speculative Generality:
  - “Oh I think we need the ability to do this kind of thing someday”
- Temporary Field:
  - An attribute of an object is only set in certain circumstances; but an object should need all of its attributes
- Data Class

  - These are classes that have fields, getting and setting methods for the fields, and nothing else; they are data holders, but objects should be about data AND behavior
- Refused Bequest
  - A subclass ignores most of the functionality provided by its superclass where the subclass may not pass the “IS-A” test (Liskov Substitution Violation)
- Comment-based coverup
  - Comments are sometimes used to hide bad code, symptomatic of large blocks of comments, not using automatic comment generation

Still Confused? Try This:

- Many more smells at: http://c2.com/cgi/wiki?CodeSmell
- Matching up a smell to its refactoring counterpart: http://wiki.java.net/bin/view/People/SmellsToR efactorings
- Taxonomy of code smells: http://www.tusharma.in/smells/
- An easy read: https://medium.com/@tusharma/how-to- track-code-smells-effectively-48dbf5ba659d

Refactoring Code

Shamelessly Inspired By Refactoring (Fowler), Chapters 6-10

Fowler Defined Different Categories of Refactoring

- *Composing Methods:* refactoring within a method or within an existing class
- *Moving Features Between Objects:* refactoring changing the responsibility of a class
- *Organizing Data:* refactoring to improve data structures and object linking
- *Simplifying Conditional Expressions:* encapsulation of conditions by replacing with polymorphism
- *Making Method Calls Simpler:* refactorings that make interfaces simpler
- *Dealing with Generalization:* moving methods up and down the hierarchy of inheritance by (often) applying the Factory or Template design pattern
- *Big Refactorings:* architectural refactoring to promote loose coupling or to realise a redesign via object orientation (Note: this is extremely difficult for most projects).

Refactoring reduces repetition and helps focus on objects

- Adding a new animal type, such as Amphibian, does not require revising and recompiling existing code
  - We have provided good extensibility which is less likely to break future code
- Mammals, birds, and reptiles are likely to differ in other ways, and we've already separated them out
  - Therefore we won't need more switchstatements in future
- Eliminated the 'flags' we needed to tell one kind of animal from another i.e. elegantly removed MAMMAL=0 etc.

Encapsulate Fields to retain private variables

- Un-encapsulated data is a violation of key object-oriented principles
- Use property get and set procedures to provide public access to private (encapsulated) member variables

PROBLEM

```
public class Compound {
   public List animals;
   …
}

public class MainClass {
   …
   public static void main(String[] args) {
     …
     int quantity = compound.animals.size();
}
```

Encapsulate Fields to Retain Private Variables

- Un-encapsulated data is a violation of key object-oriented principles.
- Use property `get` and set `procedures` to provide public access to private (encapsulated) member variables.

Problem

```
public class Compound {
    public List animals;
    …
}

public class MainClass {
    …
    public static void main(String[] args) {
        …
        int quantity = compound.animals.size();
}
```

Refactored

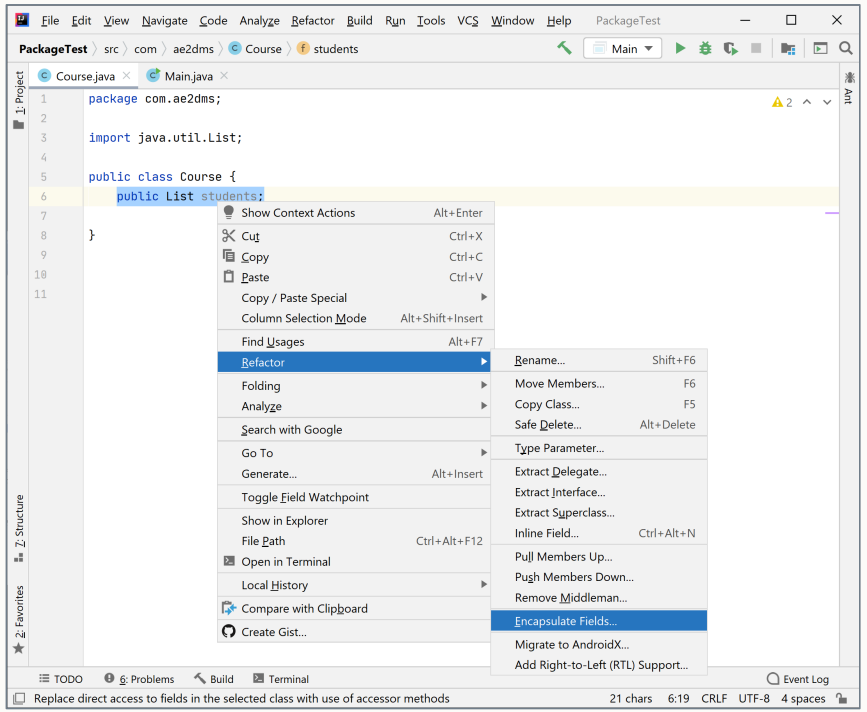
```
public class Compound {
    private List animals;

    public List getAnimals() { return animals; }
    …
}

public class MainClass {
    …
    public static void main(String[] args) {
        …
        int quantity = compound.getAnimals().size();
}
```

Encapsulating Fields Automatically With IDEs

- I have a class with 10 fields, its quite an overhead to manage and is a pain to have to go through to add get_() and set_() for each one
- Refactoring Tools
  - ADD link
- Also allows for automating two other refactoring tasks
  - **Rename Method**: cascading the new name of the method
  - **Change Method Parameters:** auto update when parameters are modified



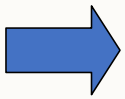
Extract Method From A Larger Block of Code

- If a method is too long or needs excessive code to describe its function, then probably extract a new method out of the code.
- Short, well-named methods are easier to maintain.
- Obeys the single responsibility principle.
- How big is too big?
  - Used to be around 125 LOC in Fortran.
  - Now have 80 lines of code (from the size of the UNIX terminal).
- Length is not the issue – it is down to “the semantic distance between a method name and the method body”.

Using "Extract Method" is a quick win!

- What are the code smells? See how the comment becomes the method name…

```
public class Customer
{
  void int foo()
  {
     …
     // Compute score
    score = a*b+c;
    score *= xfactor;
  }
}
```



```
public class Customer
{
  void int foo()
  {
     …
    score = ComputeScore(a,b,c,xfactor);
  }

  int ComputeScore(int a, int b, int c, int x)
  {
     return (a*b+c)*x;
  }
}
```

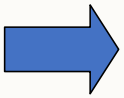
Create A New Method to Perform Method Extraction

1. Create a new method and name it after the intention of the method.
    - If can't derive this name, probably should not be extracting the method!
2. Copy the extracted code from the source method into the new method.
3. Scan the extracted code for references to local and temporary variables.
4. Check if any of the local variables are modified by the extracted code.
    - See if this can be transformed into a query instead of using the "Replace Temporarily with Query Refactor (more on this later).
5. Pass into new method as any parameters required local scope variables.
6. Replace the extracted code with a method call to the new method, checking the use of temporarily variables.
7. Run the regression tests.

Replace Parameter with Explicit Method

- You have a method that runs different code depending on the values of an enumerated parameter. Create a separate method for each value of the parameter.

```
void setValue (String name, int value) {
  if (name.equals("height")) {
   height = value;
   return;
  }
  if (name.equals("width")) {
    width = value;
    return;
  }
  Assert.shouldNeverReachHere();
}
```



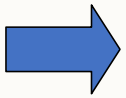
```
void setHeight(int arg)
{
    height = arg;
}

void setWidth (int arg)
{
    width = arg;
}
```

Inline Method to reduce indirection

- The method does in the body precisely the method intention

```
class PizzaDelivery
{
        // ...
   int getRating(){
      return moreThanFiveLateDeliveries() ? 2 : 1;
   }
   boolean moreThanFiveLateDeliveries(){
      return numberOfLateDeliveries > 5;
   }
}
```



```
class PizzaDelivery
{
 // ...
   int getRating() {
      return numberOfLateDeliveries
                  > 5 ? 2 : 1;
   }
}
```

Replace Temp with Query

- You place the result of an expression in a local variable for later use in your code??
- Move the entire expression to a separate method and return the result from it.
- Query the method instead of using a variable. Incorporate the new method in other methods, if necessary.

```
double calculateTotal(){
   double basePrice = quantity * itemPrice;
   if (basePrice > 1000){
      return basePrice * 0.95;
   }
   else
      return basePrice * 0.98;
}
```

```
double calculateTotal() {
   if (basePrice() > 1000) {
      return basePrice() * 0.95;}
   else {
      return basePrice() * 0.98;}
}
double basePrice() {
   return quantity * itemPrice;
 }
```

Remove Assignments to Parameters

- Some values are assigned to a parameter inside method's body, therefore use a local variable instead of a parameter.
- If a parameter is passed by reference, then after the parameter value is changed inside the method, this value is passed to the argument that requested calling this methods.
  - This occurs accidentally and leads to unfortunate effects.
  - Even if parameters are usually passed by value (and not by reference) in the programming language, this coding quirk may alienate those who are unaccustomed to it.
- Also, multiple assignments of different values to a single parameter make it difficult too now what data should be contained in the parameter at any particular point in time, makes testing harder too.
- Each element of the program should be responsible for only one thing. This makes code maintenance much easier going forward, since you can safely replace code without any side effects.
- This refactoring helps to extract repetitive code to separate methods.

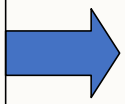
Remove assignments to parameters

- Each element of the program should be responsible for only one thing. This makes code maintenance much easier going forward, since you can safely replace code without any side effects
- This refactoring helps to extract repetitive code to separate methods

Remove assignments to parameters

- Create a local variable and assign the initial value of your parameter
- In all method code that follows this line, replace the parameter with your new local variable

```
int discount (int inputVal, int quantity)
{
        if (inputVal > 50)
        {
                inputVal -= 2;
        }
 // ...
}
```

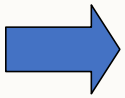


```
int discount (int inputVal, int quantity)
{
        int result = inputVal;
        if (inputVal > 50)
        {
                result -= 2;
        }
 // ...
}
```

Replace Magic Number with Symbolic Content

- Used if you have a literal number with a particular meaning
- Create a constant, name it after the meaning and replace the number with it
- This works if this is a universal constant in your code! (global variables are bad)

```
double energy (double mass, double height)
{
        return mass*height*9.81;
 // ...
}
```



```
static final double GRAVITY = 9.81;
double potentialEnergy(double mass,
double height) {
        // …
        return mass * height * GRAVITY;
 }
```

Still confused? Try this: https://www.youtube.com/watch?v=owFFVQYW1p8

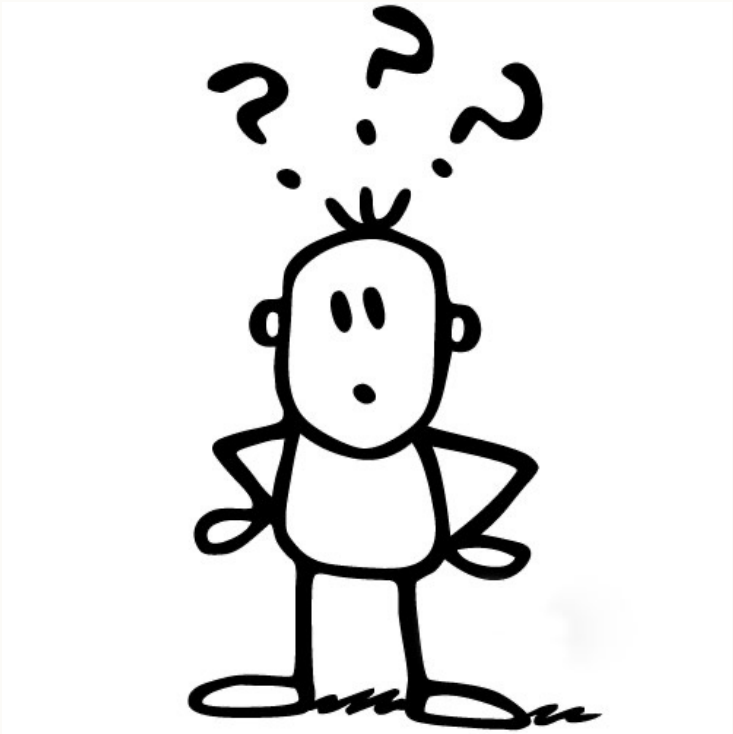
Refactoring At A Higher Level

Designing classes and really refactoring to be clean and object-oriented

Replace Type Code with Polymorphism

- `switch` statements are very rare in properly designed object-oriented code, much more common in procedural programs.
  - Therefore, a `switch` statement is a simple and easily detected “bad smell”.
  - Of course, not all uses of `switch` are bad.
  - A `switch` statement should **NOT** be used to distinguish between various kinds of object.
  - Lengthy to test all of the test cases.
- There are several well-defined refactoring for this case.
  - The simplest is the creation of subclasses.

Improvement Over `switch` using `extends`



Refactoring Reduces Repetitions and Helps Focus on Objects

- Adding a new employee type, such as Manager, does not require revising and recompiling existing code.
  - This have provided good extensibility which is less likely to break future code.
- Manager, engineer, and salesman are likely to differ in other ways, and this have already separated them out.
  - Therefore, don't need more `switch` statements in future.
- Eliminated the "flag" we needed to tell one kind of employee from another, i.e., elegantly remove `ENGINEER = 0` etc.
- Use Objects the way they were meant to be used and give our code better structure.

Single Responsibility with Extract Class

- When one class does the jobs of two, it promotes tight coupling.
- Break one class into two, e.g., having the feather details as part of the Bird class is not a realistic OO model and breaks the Single Responsibility design principle.
  - Can refactor this into two separate classes, each with appropriate responsibility.

Generalization with Extract Interface

- Extract an interface from a class. A manager want to know an Employee's role, while other managers may only need to know that certain objects can be serialized to XML. Having `toXml()` as part of the Employee interface breaks the Interface Segregation design principle which tells us that it's better to have more specialized interfaces than to have one multi-purpose interface.
- How to do this:
  1. Create an empty interface.
  2. Declare common operations in the interface.
  3. Declare necessary classes as implementing the interface.
  4. Change type declarations in the client code to use the new interface.

Push Down / Pull Up of Methods

- Eliminating duplicate behavior is a core value of refactoring.
  - “breeding ground” for bugs.
  - Whenever there is duplication, there is the risk that will alter one instance and not the other one.
- Push down to specialize a class to stop a method implemented in a superclass which is only used by one or small number of subclasses.

Push Down / Pull Up of Methods

- "Pull Up" when have methods of same composition.
  - Often after have refactored a bit to reduce the temporary variables.
  - Use when have a subclass which overrides a superclass method yet does the same thing or after applying Substitute Algorithm to make them identical.
  - If have two methods which are similar but not identical, use "Form Template" method.

https://www.refactoring.com/catalog/formTemplateMethod.html

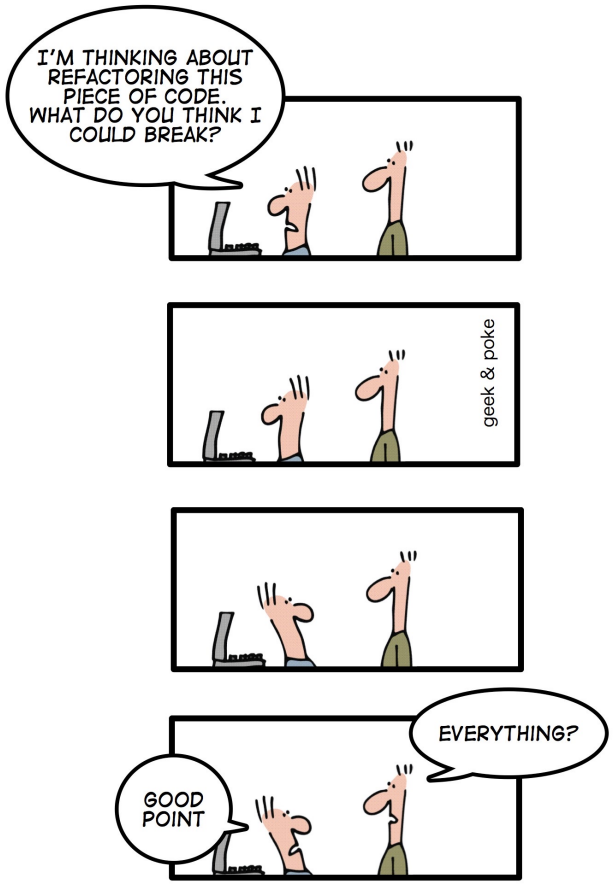
- Applies to the refactoring of a module hierarchy where a method is duplicated on two or more classes, moving the method up the hierarchy.

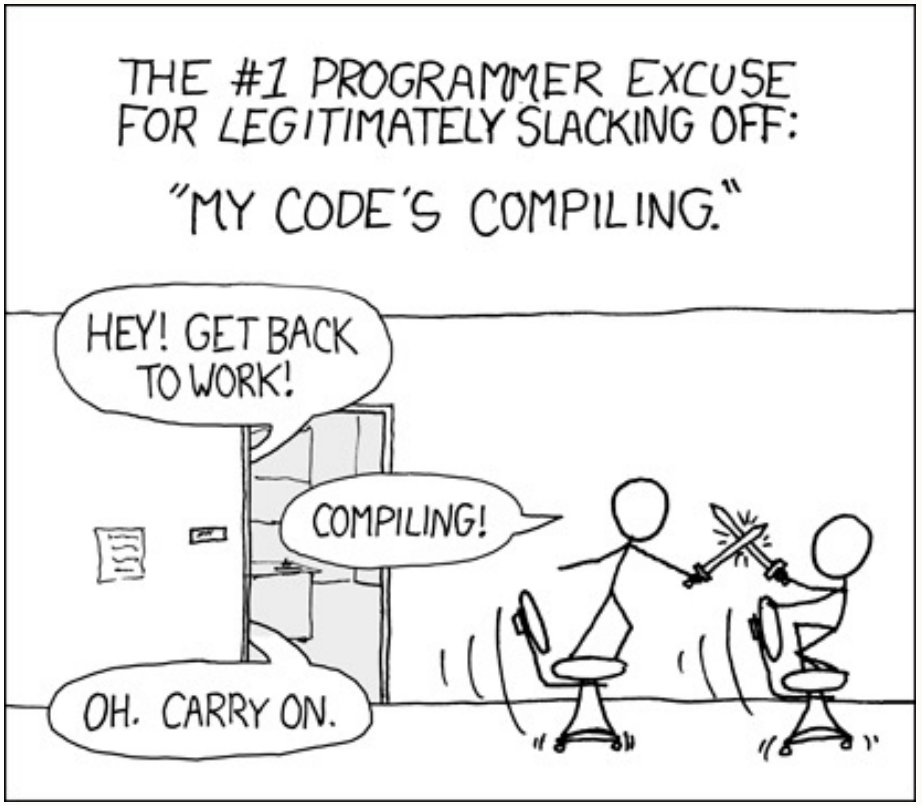
Inherit If Needs To Do with Extract Subclass

- Have found a class has features (attributes and methods) that would only be useful in specialised instances, can create a specialised of that class and give the class those features.
- This makes the original class less specialised (i.e., more abstract), and good design is about binding to abstractions whenever possible.
- Caution – too much inheritance can promote tight coupling, may want to avoid making a large collection of subclasses.

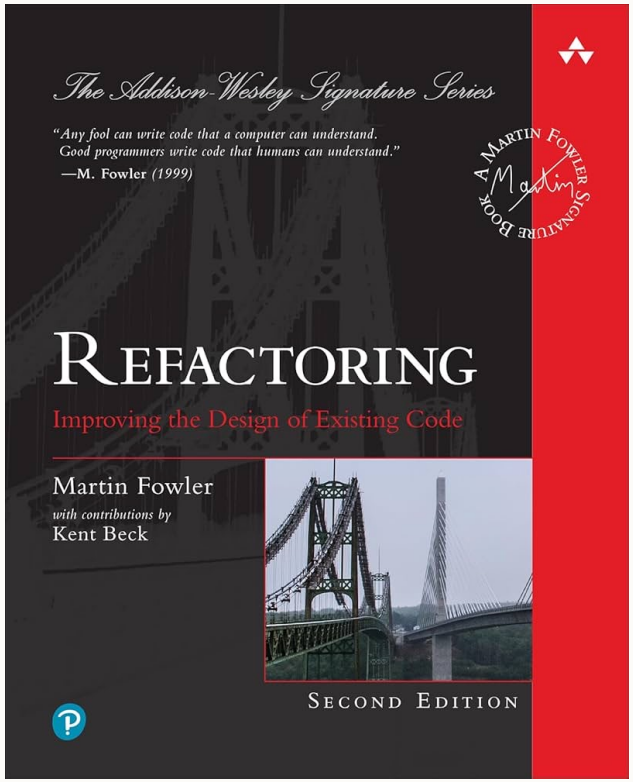
Extract Super Classes

- When there are two or more classes that share common features, consider abstracting those shared features into a superclass.
- This make it easier to produce an abstraction and removes duplicate code from the original classes.
- Not used if superclass already exists.


I'M THINKING ABOUT REFACTORING THIS PIECE OF CODE. WHAT DO YOU THINK I COULD BREAK?
geek & poke
EVERYTHING?
GOOD POINT


THE #1 PROGRAMMER EXCUSE FOR LEGITIMATELY SLACKING OFF:
"MY CODE'S COMPILING."
HEY! GET BACK TO WORK!
COMPILING!
OH. CARRY ON.

Textbooks


The Addison-Wesley Signature Series
"Any fool can write code that a computer can understand.
Good programmers write code that humans can understand."
—M. Fowler (1999)
A Martin Fowler Signature Book
Martin
REFACTORING
Improving the Design of Existing Code
Martin Fowler
with contributions by
Kent Beck
SECOND EDITION

This Week's Labs

Questions?


HOW?
WHAT?
WHERE?
why?
how?
WHO?
WHEN?
when?
HOW?
WHY?
WHAT?
when?
WHERE?
WHAT?
WHO?
WHY?
how?
how?
WHEN?
WHERE?
HOW?
who?
WHEN?
HOW?
what?
what?
WHEN?
who?
HOW?
WHAT?
WHO?
where?
why?
how?
WHY?
WHEN?
how?
WHAT?
when?
where?
HOW?
WHEN?
WHAT?
WHERE?

References

- https://cullensun.medium.com/developers-shall-enjoy-refactoring-15032db50384